

UNIVERSIDADE FEDERAL DE ITAJUBÁ

# Engenharia de Dados na Prática

---

Ingestão de dados e a camada bronze

Profa. Bárbara Pimenta Caetano

# O que veremos hoje

- 1 O que é ingestão e o que é a camada bronze

---
- 2 Proveniência: de onde veio e quando

---
- 3 Ingestão automatizada da sua fonte

---
- 4 Ingestão de uma segunda fonte por API

# Onde estamos



**Bronze**

É aqui que vamos trabalhar hoje



**Prata**

Limpeza e integração



**Ouro**

Modelagem e entrega

**Ao fim da aula, a sua camada bronze será construída por código, e não pela sua mão.**

# O que é ingestão de dados



Trazer o dado de onde ele nasce para dentro do ambiente onde ele será trabalhado, sem alterar o conteúdo.

A palavra que importa é sem alterar. Corrigir, renomear ou converter já é transformação, e isso pertence à camada seguinte.

# A bronze não é uma pasta de CSV

Ela guarda o formato em que o dado saiu da fonte, seja ele qual for.

|                     |                        |
|---------------------|------------------------|
| ■ Planilhas         | CSV, XLSX              |
| ■ Respostas de API  | JSON, XML              |
| ■ Dados geográficos | GeoJSON, shapefile     |
| ■ Extração de banco | dump, tabela exportada |
| ■ Documentos        | PDF, imagens           |

**Converter formato é transformação. Não acontece aqui.**

# Então o que a bronze é

A área onde o dado chega e fica intacto, organizada por fonte e por data de extração.

```
dados/bronze/  
  spotify/  
    artists_20260820.csv  
    proveniencia.json  
  banco_mundial/  
    paises_20260820.csv  
    proveniencia.json
```

## ■ Fiel

Mesmo formato e conteúdo

## ■ Datada

Cada extração preservada

## ■ Rastreável

Toda pasta diz de onde veio

Uma pasta por fonte. Assim duas fontes nunca se misturam.

# Por onde o dado entra

Os canais mais comuns. A lista não é fechada.

|   |                                    |                              |
|---|------------------------------------|------------------------------|
| ■ | <b>Arquivo baixado ou recebido</b> | CSV, XLSX, JSON              |
| ■ | <b>API com autenticação</b>        | Kaggle, sistemas internos    |
| ■ | <b>API aberta</b>                  | Banco Mundial, IBGE          |
| ■ | <b>Banco de dados</b>              | consulta direta a um SGBD    |
| ■ | <b>Raspagem de página</b>          | quando não existe API        |
| ■ | <b>Fila de eventos</b>             | dado que chega continuamente |

**Hoje vocês vão usar as duas formas de API.**

# O que é proveniência

O registro de onde um dado veio, quando foi obtido e por qual caminho passou até chegar aqui.

É o que permite rastrear qualquer número de um relatório final de volta até a fonte que o produziu.

Não é invenção desta disciplina. É exigência de padrões internacionais de gestão de dados, e requisito em publicação científica e em auditoria.

# De onde vem essa exigência

Os princípios FAIR orientam a gestão de dados científicos desde 2016. Hoje são cobrados por agências de fomento e por periódicos.

|          |                      |                                              |
|----------|----------------------|----------------------------------------------|
| <b>F</b> | <b>Localizável</b>   | o dado pode ser encontrado por quem procura  |
| <b>A</b> | <b>Acessível</b>     | pode ser obtido, com regras de acesso claras |
| <b>I</b> | <b>Interoperável</b> | pode ser combinado com dados de outra origem |
| <b>R</b> | <b>Reutilizável</b>  | pode ser usado de novo, por outra pessoa     |

**A proveniência está dentro do R: o item R1.2 exige proveniência detalhada.**

*Wilkinson et al. The FAIR Guiding Principles. Scientific Data, 2016. Ver também W3C, PROV-DM, 2013.*

# Na prática, três perguntas

Daqui a três meses, você vai olhar um arquivo na bronze e precisar responder:

- 1 De onde este arquivo veio?
- 2 Quando ele foi extraído?
- 3 Qual versão da fonte ele representa?

**Se a resposta depender da sua memória, o pipeline não é confiável.**

# A extração de hoje não apaga a de ontem

Cada extração vira um arquivo novo, com a data no nome.

```
dados/bronze/spotify/  
  artists_20260820.csv  
  artists_20260917.csv  
  proveniencia.json
```

**Se a fonte mudar, você consegue provar o que existia na data em que rodou a sua análise.**



# Agora vamos fazer juntos

Uma função por vez, rodando o script a cada passo.

## PASSO 1

# Instalar as novas bibliotecas

Acrescente as duas linhas ao requirements.txt:

```
pandas  
kagglehub  
requests
```

Com o ambiente virtual ativado, instale:

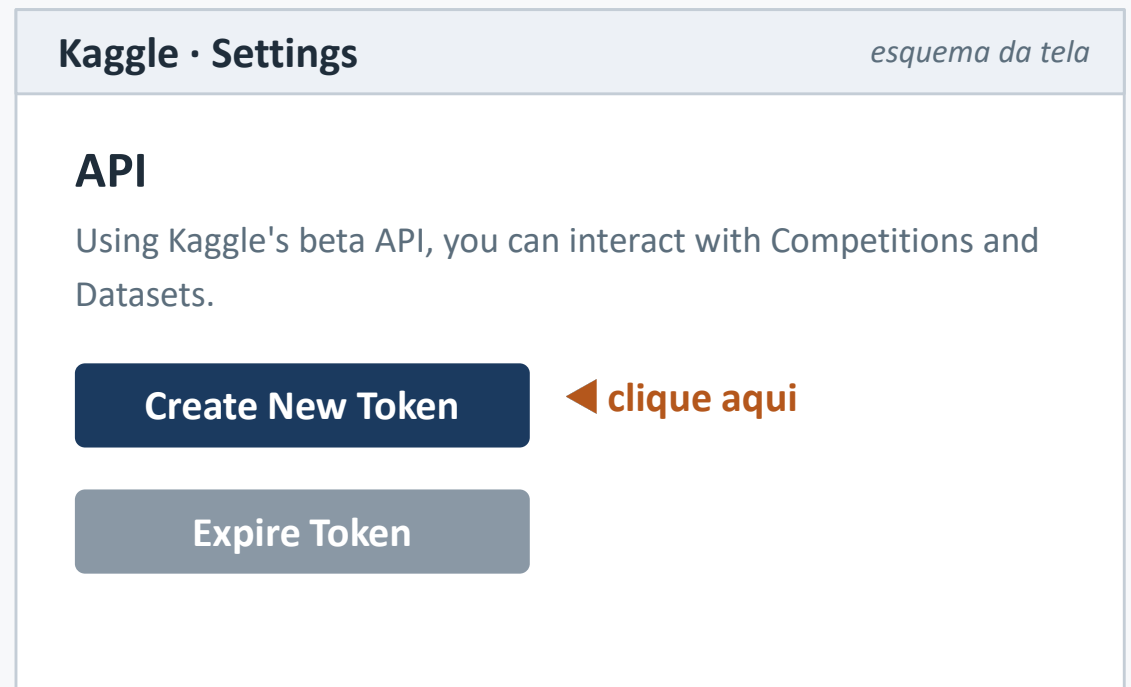
```
pip install -r requirements.txt
```

## PASSO 2

# Obter o token do Kaggle

- Entre em kaggle.com
- Abra Settings na sua conta
- Desça até a seção API
- Clique em Create New Token

O navegador baixa um arquivo chamado kaggle.json.



### PASSO 3

## Guardar o token com segurança

Coloque o kaggle.json na pasta que o kagglehub procura:

### Windows:

```
C:\Users\SEU_USUARIO\.kaggle\kaggle.json
```

### Linux ou macOS:

```
~/ .kaggle/kaggle.json
```

**Esse arquivo é a sua senha. Ele nunca entra no repositório, e já está bloqueado no .gitignore.**

## PASSO 4

### Primeira função: baixar

```
from pathlib import Path
import kagglehub

DATASET = "rishavsvault/most-streamed-artists-on-spotify"
BRONZE = Path("dados/bronze/spotify")

def baixar():
    pasta = kagglehub.dataset_download(DATASET)
    print("baixado em:", pasta)
    return Path(pasta)

if __name__ == "__main__":
    baixar()
```

Rode agora: `python src/ingerir.py`

## PASSO 5

### Segunda função: localizar o arquivo

```
def localizar(pasta):  
    arquivos = list(pasta.glob("*.csv"))  
    if not arquivos:  
        raise FileNotFoundError("nenhum CSV")  
    print("encontrados:", [a.name for a in arquivos])  
    return arquivos[0]  
  
if __name__ == "__main__":  
    pasta = baixar()  
    localizar(pasta)
```

**Nunca escreva o nome do arquivo na mão. A fonte pode renomeá-lo sem avisar.**

## PASSO 6

### Terceira função: copiar com a data

```
from datetime import date
import shutil

def copiar(origem):
    BRONZE.mkdir(parents=True, exist_ok=True)
    hoje = date.today().strftime("%Y%m%d")
    destino = BRONZE / f"artists_{hoje}.csv"
    shutil.copy(origem, destino)
    return destino

if __name__ == "__main__":
    origem = localizar(baixar())
    print(copiar(origem))
```

## PASSO 7

### Quarta função: registrar a proveniência

```
import json
from datetime import datetime

def registrar(origem, destino):
    info = {
        "fonte": DATASET,
        "arquivo_origem": origem.name,
        "arquivo_bronze": destino.name,
        "extraido_em": datetime.now().isoformat(),
    }
    (BRONZE / "proveniencia.json").write_text(
        json.dumps(info, indent=2))
```

Os três campos respondem exatamente as três perguntas da proveniência.

# Um problema no registro que acabamos de escrever

O `write_text` sobrescreve. Na segunda extração, o registro da primeira desaparece.

Isso contraria a regra da camada: a extração de hoje não apaga a de ontem.

A correção: nunca reescrever, apenas acrescentar uma linha.

```
def registrar(origem, destino):  
    info = { ... }  
  
    caminho = BRONZE / "proveniencia.jsonl"  
    with caminho.open("a", encoding="utf-8") as f:  
        f.write(  
            json.dumps(info, ensure_ascii=False)  
            + "\n")
```

## PASSO 8

### Juntar tudo em main

```
def main():  
    pasta = baixar()  
    origem = localizar(pasta)  
    destino = copiar(origem)  
    registrar(origem, destino)  
  
if __name__ == "__main__":  
    main()
```

Cada função faz uma coisa só. O main conta a história do pipeline em cinco linhas.

## PASSO 9

# Segundo script: chamar a API aberta

Crie src/ingerir\_paises.py. O Banco Mundial não exige cadastro.

```
from pathlib import Path
import requests

URL = "https://api.worldbank.org/v2/country"
BRONZE = Path("dados/bronze/banco_mundial")

def buscar():
    r = requests.get(URL, params={
        "format": "json", "per_page": 300}, timeout=30)
    r.raise_for_status()
    return r.json()
```

*Banco Mundial, Indicators API v2. Acesso público, sem autenticação.*

## PASSO 10

# Olhar a resposta antes de usar

```
def main():  
    dados = buscar()  
    print(type(dados), len(dados))  
    print(dados[0])  
  
if __name__ == "__main__":  
    main()
```

## Saída:

```
<class 'list'> 2  
{ 'page': 1, 'pages': 1,  
  'per_page': '300', 'total': 296 }
```

**Não é uma lista de países. São dois elementos: metadados e dados.**

## PASSO 11

### Quinta função: conferir se veio tudo

Agora que você viu as chaves, use-as:

```
def conferir(dados):  
    meta = dados[0]  
    print("registros:", meta["total"])  
    print("paginas  :", meta["pages"])  
    if meta["pages"] > 1:  
        print("ATENCAO: falta paginar")  
    return meta  
  
def main():  
    dados = buscar()  
    conferir(dados)
```

**Toda API pagina. Sem conferir, você trabalha com um pedaço dos dados sem perceber.**

## PASSO 12

### Sexta função: salvar e observar

```
import pandas as pd
from datetime import date

def salvar(dados):
    BRONZE.mkdir(parents=True, exist_ok=True)
    paises = pd.json_normalize(dados[1])
    hoje = date.today().strftime("%Y%m%d")
    destino = BRONZE / f"paises_{hoje}.csv"
    paises.to_csv(destino, index=False)

    print(paises["region.value"].unique())
    return destino

def main():
    ...
    salvar(dados)
```

# O que esse print revela

```
['Latin America & Caribbean ', 'Aggregates',  
'Middle East, North Africa, Afghanistan & Pakistan',  
'Sub-Saharan Africa ', 'Europe & Central Asia',  
'East Asia & Pacific', 'South Asia', 'North America']
```

- **'Aggregates' não é uma região** é a marca dos registros que não são países
- **Duas regiões têm espaço no fim** o mesmo defeito do 'Artist Type' da outra fonte

Para ver quem são os agregados, acrescente:

```
print(países[países["region.value"] == "Aggregates"]  
      ["name"].head())
```

## PASSO 13

### Sétima função: registrar a proveniência

```
import json
from datetime import datetime

def registrar(destino, meta):
    info = {
        "fonte": URL,
        "arquivo_bronze": destino.name,
        "registros": meta["total"],
        "extraido_em": datetime.now().isoformat(),
    }
    caminho = BRONZE / "proveniencia.jsonl"
    with caminho.open("a", encoding="utf-8") as f:
        f.write(json.dumps(info) + "\n")
```

Mesmo formato incremental do primeiro script. Toda fonte registra proveniência.

## PASSO 14

### Fechar o segundo script

```
def main():  
    dados = buscar()  
    meta = conferir(dados)  
    destino = salvar(dados)  
    registrar(destino, meta)  
  
if __name__ == "__main__":  
    main()
```

Os dois scripts agora têm o mesmo desenho: funções pequenas e um main que conta a história.

## PASSO 15

# Documentar as fontes no README

### ## Fontes de dados

| Fonte         | Formato | Acesso | Extraído   | Link                  |
|---------------|---------|--------|------------|-----------------------|
| Spotify       | CSV     | token  | 20/08/2026 | kaggle.com/...        |
| Banco Mundial | JSON    | aberto | 20/08/2026 | api.worldbank.org/... |

### ## Defeitos conhecidos das fontes

- A API do Banco Mundial devolve agregados regionais junto com os países.

O link é o que permite outra pessoa chegar à mesma fonte que você usou.

## ÚLTIMO PASSO

### O que subir para o repositório

```
git add .  
git commit -m "Ingestao automatizada da bronze"  
git push
```

#### Sempre

Scripts, proveniência e README

#### Depende

Os arquivos de dados da bronze

Fonte estável e versionada: o script basta. Fonte que muda sem aviso, como esta: guarde o arquivo.